Institut Supérieur d'informatique et de Mathématique de Monastir		Ministère de l'Enseignement Supérieur et de Recherche Université de Monastir
Responsable de cours : Dr. OMHENI Nizar	**Examen Session Contrôle Apprentissage Machine**	Duré Horaire : 1h30 / nb page : 3
Auditoire : L3Info		Année Universitaire : 2024-2025

Exercice 1 (10 points) :

Soit l'ensemble D des entiers suivants :

D= {2, 5, 8, 10, 11, 18, 20}

On veut répartir les données de D en trois (3) clusters, en utilisant l'algorithme K-means.

1. Appliquez K-means en choisissant comme centres initiaux des 3 clusters respectivement : 8, 10 et 11. Montrez toutes les étapes de calcul.
2. Donnez le résultat final et précisez le nombre d'itérations qui ont été nécessaires.
3. Peut-on avoir un nombre d'itérations inférieur pour ce problème ? Discutez.

Exercice 2 (5 points) :

On a un dataset **X** représentant deux caractéristiques (par exemple, heures d'étude et nombre d'heures de sommeil) et un vecteur **y** représentant les scores obtenus à un test. L'objectif est de prédire le score basé sur ces deux caractéristiques. Le code ci-dessous, écrit en Python, est destiné à exécuter le modèle décrit. Ce code comporte des erreurs que vous devez corriger afin de pouvoir lancer le modèle correctement.

1. Corriger chaque erreur identifiée
2. Expliquer votre correction pour chaque erreur identifiée

```
import numpy as np
from sklearn.model import LinearRegression
from sklearn.metrics import mean_square_error
from sklearn.model_selection import train_test_split

# Données
X = np.array([[1, 5], [2, 4], [3, 3], [4, 2], [5, 1]])
y = np.array([5, 4, 3, 2, 1])

# Diviser les données
X_train, X_test, y_train, y_test = train_test_split(X, y, test=0.2)

# Initialisation du modèle
model = LinearRegression()

# Entraînement
model.fit(X_train, y_train)

# Prédiction
predictions = model.predict(X_test)

# Calcul de l'erreur
error = mean_square_error(y_test, predictions)
print(f"Erreur quadratique moyenne: {error}")
```

Exercice 3 (5 points) :

Supposons que vous ayez un ensemble de données représentant des heures d'études vs les scores obtenus à un test. L'objectif est de prédire le score obtenu par un étudiant basé sur le nombre d'heures qu'il étudie.

1. Compléter les parties du code manquantes dans les blocs « À COMPLÉTER ».
2. Expliquer l'utilité des blocs que vous avez ajoutés.

```python
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt

# Heures d'étude
X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9]).reshape(-1, 1)

# Scores obtenus
y = np.array([51, 55, 77, 80, 82, 85, 87, 90, 95])

# Diviser le dataset en ensemble d'apprentissage et de test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Initialiser le modèle de régression linéaire
model = LinearRegression()

# Entraîner le modèle avec les données d'entraînement
# À COMPLÉTER

# Prédire les scores pour les données de test
# À COMPLÉTER

# Afficher la pente (coef_) et l'ordonnée à l'origine (intercept_) du modèle
print(f"La pente: {model.coef_}")
print(f"L'ordonnée à l'origine: {model.intercept_}")

# Tracer les données et la ligne de régression
plt.scatter(X, y, color='blue') # Données réelles
plt.plot(X_test, # À COMPLÉTER, color='red') # Ligne de régression
plt.title('Heures d'étude vs Score')
plt.xlabel('Heures d'étude')
plt.ylabel('Score')
plt.show()
```